

PLAYBOOK DE ENGENHARIA: METODOLOGIA ONDA-DEV

1. INTRODUÇÃO E FILOSOFIA

Este documento estabelece o processo padronizado para o ciclo de vida de desenvolvimento de software da Onda[cite: 1]. A metodologia foi desenhada para o cenário de um **Desenvolvedor Solo** operando com Inteligência Artificial de alta performance (Antigravity como IDE/Orquestrador e Claude Code no terminal).

Baseamo-nos nos princípios do **Agile Vibe Code**[cite: 3] e do **Extreme Programming (XP) com IA**[cite: 2]. Substituímos a burocracia focada em gestão de pessoas por uma **Iteratividade de Engenharia**, garantindo adaptação a mudanças, entregas contínuas e altíssima qualidade técnica.

2. A DINÂMICA ÁGIL E ENTREGÁVEIS

Trabalhando de forma solo acelerada por IA, o formato descarta "Sprints" de tempo fixo que geram prazos artificiais.

- **O Modelo de Fluxo (Kanban):** Trabalhamos com fluxo contínuo. Você puxa a próxima tarefa apenas quando a atual está testada e entregue.
- **Os Entregáveis (Milestones):** A previsibilidade baseia-se em gerar entregáveis palpáveis em cada etapa, comprovando o avanço para o cliente e para o negócio:
 - **Fase 1:** A *Spec Viva* (Arquivo `.claudemd` com Histórias de Usuário)[cite: 2].
 - **Fase 2:** O *Protótipo Estático* (Interface visual online para aprovação).
 - **Fase 3:** O *Blueprint* (Arquivo `ROADMAP.md` e Contratos de API).
 - **Fase 4:** *Small Releases* (Commits e módulos testados no repositório)[cite: 2].
 - **Fase 5:** *Software em Produção* (Deploy rodando na nuvem).

3. O MACROFLUXO DE EXECUÇÃO (AS 5 FASES)

A execução obedece estritamente à seguinte linha de montagem:

FASE 1: INÍCIO — A ESPECIFICAÇÃO VIVA (**CLAUDE.md**)

Transforma requisitos do cliente em fatias de valor e no "cérebro documental" do projeto[cite: 2].

1. **Mapeamento de Épicos:** O sistema é dividido em grandes blocos (ex: *Épico 1: Gestão de Assinaturas*).
2. **Histórias de Usuário:** Cada Épico é quebrado no formato:

***Como** [tipo de usuário], **eu quero** [realizar uma ação] **para que** [obter algum benefício].*

3. **Critérios de Aceite:** Regras rigorosas de sucesso.

Dado que o usuário está na tela de pagamento, quando ele insere um cartão inválido, então o sistema deve barrar e exibir a mensagem de erro.

4. **Criação do **.claudemd**** : A IA gera este arquivo na raiz contendo os Épicos, Histórias, Stack e Arquitetura. **Ele é a única fonte da verdade e evolui com o projeto**[cite: 2].

FASE 2: LAYOUT E CONGELAMENTO VISUAL

Mitiga o risco de o cliente alterar o fluxo e destruir o banco de dados no futuro.

1. **Geração Estática:** A IA lê o **.claudemd** e gera o front-end 100% estático com dados fictícios.
2. **Aprovação:** O layout é apresentado online para o cliente validar.
3. **Congelamento Visual:** Aprovado, o layout está congelado. A engenharia tem luz verde. Mudanças futuras exigem renegociação de prazos.

FASE 3: BLUEPRINT DE ARQUITETURA E CONTRATOS (**ROADMAP.md**)

1. **Modelagem:** Com o visual aprovado, as tabelas de banco de dados e relacionamentos são definidos com exatidão.
2. **Geração do Roteiro:** O Antigravity gera o **ROADMAP.md** , dividindo o código em Módulos Independentes e Tarefas.
3. **Contratos (API-First):** As assinaturas de *Request/Response* (JSON) são definidas no roadmap antes de iniciar o código.

FASE 4: A ESTEIRA EXTREME PROGRAMMING (Codificação)

Execução pesada no terminal (Claude Code), consumindo o **ROADMAP.md** sequencialmente. Lembre-se: cerca de 37% dos commits serão features novas, o restante é segurança, refatoração e testes[cite: 2].

1. **A Diretiva Primária:** O primeiro comando inegociável na sessão do terminal:

*"Leia o **.claudemd** e o **ROADMAP.md** . A partir de agora, não altere a sintaxe do código que eu enviar ou que já existe. Este é o padrão a ser seguido adiante."*

2. **O Ciclo de TDD Iterativo (Test-Driven Development)[cite: 2]:**

- **Passo A (Red - Mocks):** A IA cria os testes abrangentes usando **Mocks** (simulando o banco de dados/APIs para isolar a lógica). O teste **deve falhar**, pois a lógica real ainda não foi escrita.
- **Passo B (Green - Código):** A IA escreve o código de aplicação (a regra de negócio) estritamente necessário para fazer a luz verde do teste acender.
- **Passo C (Refactor):** Pausa higiênica. A IA aplica DRY (não se repita) e otimiza o código sem quebrar os testes.

3. **Integração Contínua:** Todo commit deve estar limpo e pronto para produção (*Small Releases*)[cite: 2].

FASE 5: HOMOLOGAÇÃO, DEPLOY E ENCERRAMENTO

1. **Smoke Test Local:** Subir o ambiente via Docker. Rodar toda a esteira de testes e realizar a validação humana de ponta a ponta.
2. **Deploy:** Envio automatizado via CI/CD para a nuvem. Entrega oficial.

4. GESTÃO DE MUDANÇAS E RETORNOS NO FLUXO

- **Retorno à Fase 1:** Se o cliente pedir uma funcionalidade nova (ex: "Adicionar PIX") durante o backend, você não codifica na hora. O fluxo retorna à Fase 1: atualize o `.claudemd` com a nova História de Usuário. A IA lê a regra, atualiza os testes, e só depois codifica[cite: 2].
- **Retorno na Fase 2:** Mudar o visual aprovado enquanto o backend está sendo feito impacta as tabelas do sistema. Caracteriza mudança de escopo e exige aditivo de prazo.

5. DIRETRIZES DE PREVISIBILIDADE E ESTIMATIVAS

Para um desenvolvedor solo com IA, estimar por "horas" é ineficaz. A métrica de prazos utiliza o **Peso dos Módulos do ROADMAP.md** (Complexidade + Risco):

- **Módulo Pequeno (~1 a 2 dias):** Código mecânico e baixo risco. Ex: CRUDs simples, telas estáticas, perfis de usuário.
- **Módulo Médio (~3 a 4 dias):** Lógica intermédia, exigindo mais atenção no TDD. Ex: Relatórios com gráficos, consumo de APIs externas.
- **Módulo Grande (~5 a 7 dias):** O "coração" do sistema, com alto risco de falha crítica. Ex: Gateway de pagamentos, algoritmos core, sistemas robustos de segurança e permissões.

A Fórmula de Prazo Técnico Final

A sua data de entrega blindada para o cliente é a soma de:

Tempo da Fase 2 (Aprovação Visual) + Soma(Dias dos Módulos Fases 3 e 4) + 2 dias (Fase 5 - Homologação)

Exemplo Prático: O projeto exige 2 módulos pequenos (4 dias) e 1 grande (7 dias). A engenharia consumirá 11 dias. Adicionando os 2 dias de deploy/homologação, você apresenta um prazo técnico exato de **13 dias úteis** para o seu sócio ou cliente.